[Javascript 강좌] 제 18강 클로저 Javascript

2019. 1. 30. 0:48

https://blog.naver.com/fb_lua/221454308062

안녕하세요.
Javascript 강좌중인 Falcon 입니다.

클로저라는 개념은 쉽게 정리하면 함수 안에 함수가 또 있는 것을 의미합니다. 즉, 외부함수와 내부 함수로 나뉘어진 것을 클로저라고 보시면 됩니다. 코드를 보면서 알려드리도록 하겠습니다.

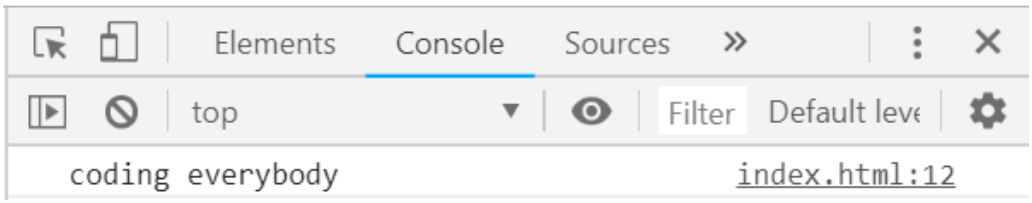
```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>클로저</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         function outter() {
10          function inner() {
11            var title = 'coding everybody';
12            console.log(title);
13          }
14          inner();
15        }
16        outter();
17      </script>
18    </body>
19  </html>
```

outter()	외부함수	노란색 네모
inner()	내부함수	하늘색 네모

위처럼 외부 함수안에 내부 함수가 존재하는 것을 클로저라고 합니다. 내부 함수에 선언된 11행의 변수는 외부함수로 나올 수 없습니다. 하지만 외부 함수에 선언된 변수는 내부 함수에서 사용할 수 있습니다.

```
7   <body>
8     <script type="text/javascript">
9       function outter() {
10        var title = 'coding everybody';
11        function inner() {
12          console.log(title);
13        }
14        inner();
15      }
16      outter();
17    </script>
18  </body>
```

10행 부분은 외부 함수에 선언된 변수이지만 12행에서 받아들이고 있는 것을 볼 수 있습니다. 또한 16행에서 호출한 것은 외부 함수이지만 외부 함수인 14행에서 inner()을 호출하기 때문에 출력이 되는 것 입니다.



위는 둘다 coding everybody라는 문자열이 출력 됩니다. 그러면 클로저의 실용적인 사용법을 익혀봅시다.

노란색 네모	외부 함수
파란색 네모	내부 함수

```
8   <script type="text/javascript">
9     function movie(title) {
10      return {
11        get_title : function() {
12          return title;
13        },
14        set_title : function(_title) {
15          title = _title;
16        }
17      }
18    }
19    ghost = movie('Ghost in the shell');
20    matrix = movie('Matrix');
21
22    console.log(ghost.get_title());
23    console.log(matrix.get_title());
24  </script>
```

위를 보면 return 반환을 하면서 객체를 반환하고 있습니다. 함수 내부에 객체를 생성하고 속성(property)값 안에 함수를 선언했다고 해도 이는 클로저에 해당 됩니다. 즉, 외부 함수와 내부 함수라는 사실은 변함이 없다는 것이죠.

```
19    ghost = movie('Ghost in the shell');
20    matrix = movie('Matrix');
```

19행과 20행의 경우 var라는 키워드가 붙지는 않았지만 이는 엄연히 변수 입니다. 그리고 이 변수 안에는 return반환되는 객체가 안에 들어가게 된다. 여러분들이 알고 계신 함수는 return 반환됨과 동시에 사라져버린다.

```
9   function func() {
10    return 1;
11  }
```

이런 함수들은 함수가 호출되고 반환되는 동시에 사라져버리지만 클로저는 반환이 되어도 사라지지 않는다.

function movie(title)

movie에서 매개변수로 받고 있는 title이 사라져버리면 내부 함수에서 선언되고 있는 title은 그 어떠한 값도 가지지 못하기 때문이다. 단 이 변수로 접근을 하기 위해서는 내부 함수만 가능하다는 사실을 알고 넘어가도록하자.

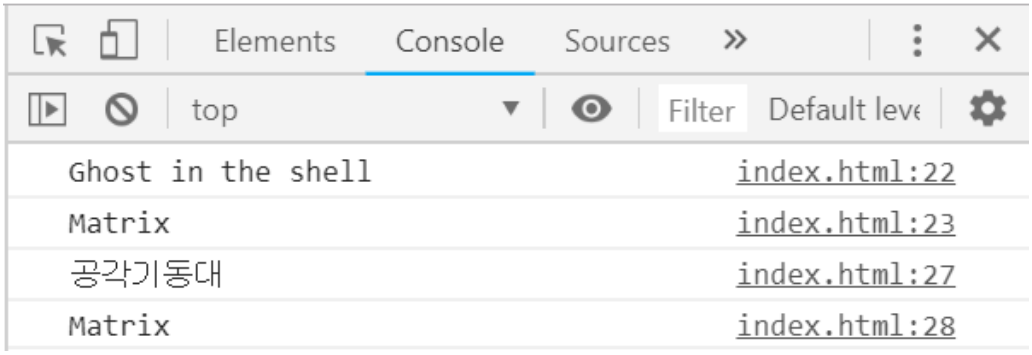
```
8         <script type="text/javascript">
9           function movie(title) {
10            return {
11              get_title : function() {
12                return title;
13              },
14              set_title : function(_title) {
15                title = _title;
16              }
17            }
18          }
19          ghost = movie('Ghost in the shell');
20          matrix = movie('Matrix');
21
22          console.log(ghost.get_title());
23          console.log(matrix.get_title());
24
25          ghost.set_title('공각기동대');
26
27          console.log(ghost.get_title());
28          console.log(matrix.get_title());
29        </script>
```

내부 함수에서만 접근이 가능하기 때문에 title의 값을 바꾸려면 내부 함수를 이용해야 합니다. 25행이 그 예를 들고 있습니다. 소스코드가 많이길어보이니 소스코드를 배포해드리도록 하겠습니다.

소스코드.txt

```
25          ghost.set_title('공각기동대');
26
27          console.log(ghost.get_title());
28          console.log(matrix.get_title());
```

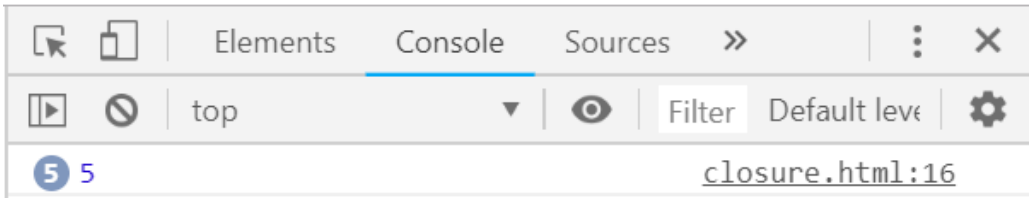
위에서도 설명해드렸지만 클로저는 반환이 되어도 함수가 사리지지 않습니다. 만약 사라졌다면 25행의 부분의 코드에서 title의 값을 변경하지 못하게 됩니다. 이 점 꼭 기억해주시기 바랍니다.



위의 코드를 실행해보면 Ghost in the shell 이란 문자열이 공각기동대로 바뀐것을 볼 수 있습니다.
지금부터 알려드릴 내용은 클로저와 관련해서 자주 언급되는 문제라고 합니다.

```
8   <script type="text/javascript">
9     var arr = [];
10    for(var i = 0; i < 5; i++) {
11      arr[i] = function() {
12        return i;
13      }
14    }
15    for(var index in arr) {
16      console.log(arr[index]());
17    }
18  </script>
```

아주 기초적인 문법이기 때문에 분석하는데는 크게 어려움은 없으실겁니다. 그런데 여기서 문제가 일어납니다.



이 소스코드를 실행하면 5라는 숫자가 5번 반복됩니다. 하지만 위의 코드에서 바라는 것은 0 ~ 4라는 숫자 입니다.
이 부분은 잘 생각하고 이해를 하셔야하는 문제 입니다.

```
10    for(var i = 0; i < 5; i++) {
11      arr[i] = function() {
12        return i;
13      }
14    }
```

쉽게 생각하시면 됩니다. 이 부분이 문제가 되는 것은 바로 for문의 var i라는 값이 전역변수이기 때문 입니다.
12행 부분의 i는 전역변수 var i를 바라보고 있습니다. for문이 다 돌고나면 i의 값은 5가 됩니다.

```
15          for(var index in arr) {
16            console.log(arr[index]());
17          }
```

이 부분에서 arr[0]을 호출하면 return i가 반환됩니다. 여기서 중요한것은 12행 return i가 바라보는 것은 이미 반복문을 통해 5까지 진행된 i의 값이라는 겁니다. 그렇기 때문에 이 부분에서 호출되는 것은 숫자 5가 5번 나오게 됩니다. 이러한 문제점을 해결할 수 있는 방법이 있습니다.

```
7       <body>
8         <script type="text/javascript">
9           var arr = [];
10          for(var i = 0; i < 5; i++) {
11            arr[i] = function(id) {
12              return function() {
13                return id;
14              }
15            }(i);
16          }
17          for(var index in arr) {
18            console.log(arr[index]());
19          }
20        </script>
21      </body>
```

여기서 중요한 것은 15행의 (i)인데 이는 함수를 바로 호출하겠다는 뜻 입니다. 즉, 이 함수가 호출되고 나서 arr이라는 배열에 들어가는 부분은 바로 이 부분입니다. 또한 호출될 당시 i의 값을 인자 값으로 넘겨주면서 11행의 id라는 매개변수가 받아버리기 때문에 i의 값이 바뀌더라도 영향을 받지 않게 됩니다.

```
12              return function() {
13                return id;
14              }
```

그리고 arr[] 배열에 들어가는 부분들인 바로 이 부분입니다. 외부 함수가 사라지지 않는다는 것을 이용해서 해결을 할 수 있습니다. 클로저에 대한 이런 문제점들이 더 있다고는 하지만 아직 저도 이 클로저를 사용해본 적이 없습니다. 저도 아직 배우면서 정리 중이라서 더 자세한 내용은 추후 경험이 생기면 더 올려드리겠습니다.

이것으로 [Javascript 강좌] 제 18강 클로저 편을
마치도록 하겠습니다.